

Territory-Aware Phase 1: Validating the Winner-Take-All Balance Term at High N

On the bad seed that was immune to raising λ , the balance term drives the entrenched runt from -34% to *zero* imbalanced cells — but the discrete-area trim removes the energy plateau, so no level triggers mesh refinement, and the coarse level is shown to be resolution-floor-limited

July 2026

Status — measured (partial: the finest level was interrupted; confirmation is solid through level 1)

This report **confirms** the Stage-6 hypothesis of `docs/plans/PHASE1_TERRITORY_AWARE_IMPLEMENTATION_PLAN.md`: on the *bad* seed and λ that produced two entrenched -34% runts under the original energy, enabling the winner-take-all balance term drives the partition to 0 imbalanced cells (worst $\approx 2\%$), holding through a mesh transition. It also records two diagnoses of *why the run was slow*: (i) the discrete-area trim's periodic retargeting removes the energy plateau, so no level fires the mesh refinement trigger — every level runs to the iteration cap; and (ii) the coarsest level is resolution-floor-limited (worst deviation floors at $\approx 10\%$, above the 5% gate), so refinement is *necessary*, not merely faster. Numbers are independently recomputed from the run's trace snapshots. **Partial:** the run died mid-level-2 (host interruption), so the formal finest-level completion gate was not reached; the confirmation rests on level 1's completed cap and a level-2 mid-snapshot.

Provenance

Date	2026-07-20 (analysis of the run of 2026-07-17... 07-19)
Surface	torus $T(R=1.0, r=0.6)$, $N=200$, seed 84172851, $\lambda=9$, 3 levels, <code>init_method: seeded</code>
Branch	<code>feat/phase1-territory-aware-relaxation</code>
Source run	territory: <code>run.20260717_102306</code> , <code>config</code> <code>parameters/torus_200part_coarse_seeded_lam9_territory_test.yaml</code> (<code>wta_balance_enabled</code> $\gamma=7.0$, $p=2$; <code>wta_trim_enabled</code> period 200, damping 0.5, clamp 0.20; <code>pgd_reduced_gradient</code> 8 dual sweeps). Control: <code>run.20260712.224424</code> (same seed/ λ , original energy, no flags).
Producing script	<code>make_figures.py</code> (beside this report; trajectory arrays are the extracted measurement — see its header)
Method inputs	per-level trace snapshots <code>traces/*_level{0,1,2}_internal_data.hdf5</code> (density field x every 500 iters); the run's <code>relaxation.log</code>
Versions	Python 3.12.13, numpy 2.4.4, scipy 1.17.1, matplotlib 3.10.8, h5py 3.16.0
Anchor	control worst-cell deviation = -34.2% (2 imbalanced); territory level-1 end = 0 imbalanced, worst $\pm 2.1\%$ (recomputed, matching <code>detect_area_imbalance</code>)

1 Question

Phase 1 pins equal area on the *continuous* density field, but the deliverable is the *argmax* (winner-take-all) partition; at high N the discrete areas drift, producing “runt” cells that make the Phase-2 equal-area constraint infeasible (`docs/reference/winner_take_all_partition_gap.md`). The territory-aware term (`docs/math/07-phase1-wta-balance/`) adds a penalty on the discrete-area imbalance plus a periodic trim that retargets the projection toward exact discrete equality. **Does it fix an entrenched runt on a seed and λ known to be immune to raising λ — the “anti-lottery” property $N=1000$ needs?**

2 Method

For each saved snapshot of the density $x \in \mathbb{R}^{V \times N}$, we rebuild that level’s torus mesh (`TorusMeshProvider`), take the winner-take-all label $w_i = \arg \max_k x_{ik}$, assign each vertex its lumped mass v_i , and form the discrete cell area $A_k = \sum_{i: w_i=k} v_i$. The relative deviation is $(A_k - \bar{A})/\bar{A}$ with $\bar{A} = \sum_i v_i/N$; a cell is *imbalanced* if $|\text{dev}| > 5\%$. This is exactly the `detect_area_imbalance` computation. All numbers below are recomputed directly from the trace snapshots.

3 Result 1 — the mechanism works on the bad seed

The control (original energy, same seed/ λ) ends with worst-cell deviation -34.2% and 2 imbalanced cells. With the territory flags on, the balance improves monotonically across levels to **0 imbalanced cells** (worst $\pm 2.1\%$) by the end of level 1, and level 2 inherits and holds it (Table 1, Fig. 1). No cell dies.

4 Result 2 — why no level triggered mesh refinement

Both completed levels ran to the 30,000-iteration cap; the log contains no “Refinement triggered” line. The cause is the trim, not a stuck optimizer. Every `wta_trim_period = 200` accepted

Table 1: Winner-take-all discrete-area imbalance, recomputed per level.

Level (verts/cell)	snapshot	worst dev	$n_{\text{imb}} (>5\%)$
Level 0 (48)	iter 0	$-30.3\%/+54.0\%$	149
Level 0 (48)	iter 29 999 (cap)	$-10.1\%/+7.0\%$	4
Level 1 (124)	iter 0 (post-interp)	$-17.9\%/+14.5\%$	51
Level 1 (124)	iter 29 999 (cap)	$-2.1\%/+2.1\%$	0
Level 2 (237)	iter 0 (post-interp)	$-4.3\%/+4.6\%$	0
Level 2 (237)	iter 7 500 (died)	$-1.5\%/+2.4\%$	0
<i>Control</i>	final	-34.2%	2

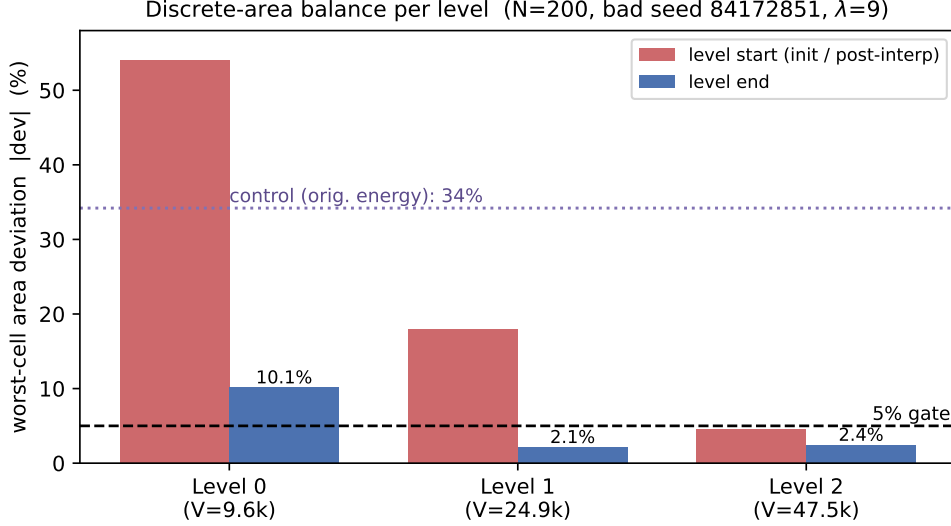


Figure 1: Worst-cell deviation at each level’s start and end vs the control. The mechanism clears the 5% gate; the control (dotted) stays at 34%.

iterations the trim retargets the per-cell area target d , re-projects, and re-baselines the energy — each retarget bumps E up before the next descent, so the energy **sawtooths and never plateaus** (level-0 tail: E oscillates $\approx 409\text{--}437$, amplitude ~ 8). The refinement trigger fires on an energy plateau ($\Delta E < 10^{-8}$ held for 30 checks); the sawtooth’s swings are nine orders of magnitude above that, so the plateau criterion is structurally impossible to satisfy. This behaviour is *new*: no standard/production config sets any of the three territory flags (verified), so previous $N=100/150/200$ runs plateaued cleanly and triggered normally. The extra iterations are not wasted spinning — the balance keeps improving — but the level cannot stop itself.

5 Result 3 — refinement is necessary, not merely faster

The trajectory of the worst deviation *within* each level (Fig. 2) settles the design question of whether one could simply run the coarsest level longer:

- **Level 0 is floor-limited.** Its worst deviation flatlines at $\approx 10\%$ — *twice* the 5% gate — with a tail rate of 0.23%/1000 iterations (pure sawtooth noise). The count of bad cells keeps falling ($14 \rightarrow 4$), but the *worst* cell is stuck, and the gate needs *every* cell under 5%. At 48 verts/cell the discretization cannot represent 200 equal areas below $\approx 10\%$; more iterations do not help.
- **Level 1 crosses the gate, steeply.** At 124 verts/cell the achievable floor drops to $\approx 2\%$; the first 5000 iterations fall at 2.2%/1000 ($\sim 10\times$ level 0’s tail), reaching 0 imbalanced by

~iter 9000.

The 5% gate lies *between* the two levels' resolution floors ($f_b \propto 1/\sqrt{V/N}$, the validity plan's band-fraction law), so refinement is what crosses it. Note the gate was crossable at just **124 verts/cell** here — the validity plan's mesh-budget floor guessed ~ 600 for margin, so ~ 250 – 300 finest may suffice (a 2–4 $\times$ compute rebate at $N=1000$, worth confirming).

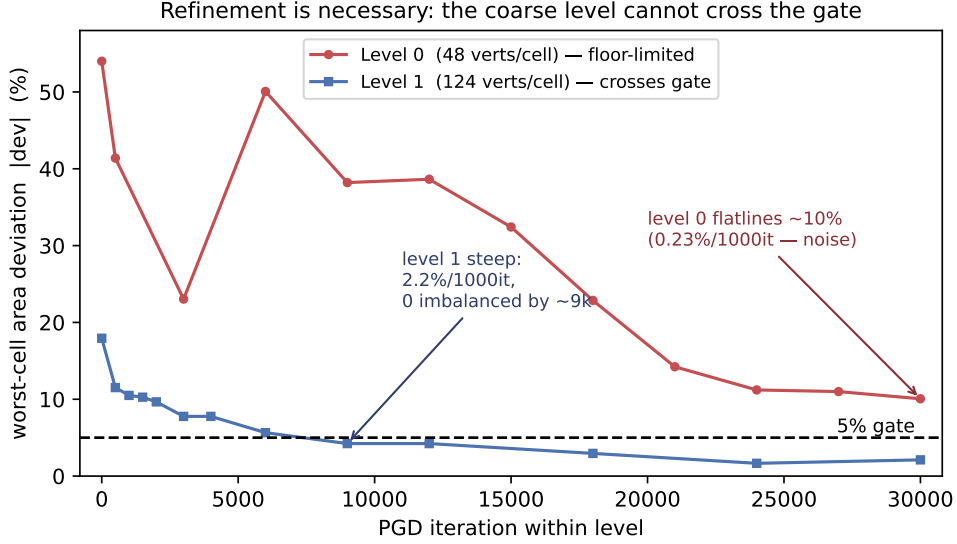


Figure 2: Worst-cell deviation vs iteration within levels 0 and 1. Level 0 floors above the gate; level 1 (finer) descends $\sim 10\times$ faster and crosses it.

6 Conclusions and implications

1. **The balance term works** on the hardest case (bad seed, λ immune) — the anti-lottery property $N=1000$ requires. This confirms Stage 6.
2. **Its cost is diagnosed, not fundamental.** The slowness (level 0 alone 9.5 h, level 1 ~ 31 h, both to the cap) is the trim sawtooth preventing early triggering plus genuinely-continued optimization — both tunable.
3. **The coarse-only schedule needs the switch *after* level 1, gated on the imbalance metric, not fixed at level 0.** Level 0 cannot clear the gate (floor); the machinery must run until a level's floor drops below the gate (level 1 here), then the cheap original energy carries the already-balanced fine levels (level 2 started at 0 imbalanced). Dropping the trim on the cheap levels also restores clean plateaus and normal triggering. See docs/plans/PHASE1_COARSE_ONLY_WTA_SCHEDULE.md.
4. **Refine on a balance/stationarity plateau, not raw energy.** Level 0's balance flatlined by $\sim$ iter 22k while the trim-masked energy kept crawling; a gate-aware trigger would have refined there, saving ~ 3 h at no cost to the result.

7 Caveats

Single seed / single λ (by design — the point is the *bad* case). The run was interrupted mid-level-2, so the formal finest-level completion gate was not reached; a cluster re-run (with checkpoint/resume, and preferably the tuned schedule of conclusion 3) is needed to close it. The next A/B is $N=300$ vs run_20260714.224821 (predicted $9 \rightarrow \leq 1$ over gate).